
W2F: One-Step Word to Face Generative Typography via Drifting

Yuxuan Chen*

Junhao Huang

Yanmohan Wang

Abstract

We present **Words to Faces (W2F)**, a one-step generative framework that composes a recognizable human face out of a fixed set of $K=12$ alphabet glyphs. A single feed-forward generator predicts, for every glyph, both *what* to draw (a small 32×32 letter image) and *where* to put it (an affine pose), and a Spatial Transformer Network (STN) places the glyphs onto a 128×128 canvas. The model uses neither a GAN discriminator nor an iterative diffusion sampler: it is trained *only* by the recently proposed **Drifting** loss [4], a particle-based distribution-matching objective applied at two levels — a per-glyph signal that keeps each placed image on its letter manifold, and a composite signal that pulls the whole canvas toward the distribution of real face line-drawings. We work in a deliberately hard corner of the design space: one-step generation, no face-specific priors, and no paired (text $\rightarrow$ face) supervision. We give a complete, reproducible description of the architecture, the two-level Drifting losses (with full pseudocode and every modification that distinguishes our variant from the original objective), and the exact mathematical form of all eleven loss terms, the two data pipelines, and the frozen models used for evaluation. Our central empirical finding is a sharp *legibility–realism trade-off*, which we characterize quantitatively, and we ablate our main design choice, feature-wise letter conditioning (FiLM).

1 Introduction

Typography art — arranging letters so that, viewed as a whole, they form a larger image such as a face — is a classic example of *emergent* visual structure. Reproducing it automatically is hard for two reasons. First, the elements are *constrained*: each must remain a recognizable letter, so the model cannot draw arbitrary strokes. Second, the global target (a face) and the local target (a letter) live in different distributions that must be satisfied *simultaneously* by the same pixels.

Most modern generative models reach such targets through machinery we explicitly want to avoid. GANs [5] introduce a discriminator that is unstable to train and prone to mode collapse; diffusion models [6] require many sequential denoising steps (which one-step distillations such as consistency models [17] only partly remove). We instead build on **Drifting** [4], which trains a *one-step* generator by moving generated samples along score-like vector fields: toward the real-data manifold (an attraction field V_p) and away from other generated samples (a repulsion field V_q). Drifting needs no discriminator and no iterative sampler.

Within this paradigm we solve the typography-face problem under three constraints that define our **design space**: (i) **one-step** generation (a single forward pass, no iterative refinement); (ii) **no priors** — no facial landmark templates, no hand-placed eye/nose/mouth anchors, and no face-engineered similarity measures (generic structural measures such as intersection-over-union, which know nothing about faces, are permitted); (iii) **no paired data** — the model never sees a (text $\rightarrow$ face) pair; it is

*Team leader

given one unpaired collection of real face line-drawings and one unpaired collection of letters, and the Drifting objective is what bridges them.

Contributions. (i) A one-step generator whose two-stage transformer is built to *co-adapt with the STN*: the predicted layout is fed both into the spatial transformer and back into the rendering stage (Sec. 4.2). (ii) A full, reproducible instantiation of Drifting at two hierarchical levels with several modifications beyond the original algorithm, given as pseudocode and exact equations (Sec. 4–4.3). (iii) A pure-drifting face-generation *baseline* that establishes the face-quality ceiling, documented end-to-end including the frozen models used for evaluation (Sec. 5.2–5.3). (iv) A quantitative characterization of the *legibility–realism trade-off* and an ablation of feature-wise letter conditioning (Sec. 5.4–5.5).

2 Related work

One-step / distribution-matching generation. Beyond GANs and diffusion, generators can be trained by directly matching distributions, e.g. Maximum Mean Discrepancy [10], optimal-transport divergences, and recently score- and particle-based “drift” objectives that push one-step samples toward the data score [16] without a learned critic [4]. Our work applies and stress-tests Drifting in a structurally constrained, compositional setting.

Typography and artistic text rendering. Generating images out of glyphs spans structure-based ASCII art [20], calligram layout, and neural semantic typography that deforms letters toward a target shape [7, 18]. These methods, however, target a *single* given word or image and optimize it per instance — by shape matching, score-distillation [14], or diffusion guidance — and the semantics they inject come from a pretrained vision-language prior. In contrast, W2F learns an *amortized, one-step* generator, injects no semantic prior, and is supervised only by distribution matching against unpaired collections, so a single forward pass turns an arbitrary letter string into a face.

Spatial Transformer Networks. STNs [8] apply differentiable affine warps to images. We use an STN as a parameter-free placement operator that keeps the whole pipeline end-to-end differentiable.

Conditioning by feature modulation. Feature-wise linear modulation (FiLM) [13] modulates intermediate features by a predicted per-channel scale and shift; we use it to inject letter identity into the decoder.

Edge maps and evaluation. We obtain face line-drawings with the Canny edge detector [2] on CelebA [11], augment letters from EMNIST [3], and evaluate with a kernel-distance metric [1] in a frozen feature space, plus improved precision/recall [9].

3 Preliminaries

The Drifting loss. For a generated sample y , Drifting [4] forms a target by taking one score-like step toward the data and away from siblings, and regresses to it under a stop-gradient:

$$\mathcal{L}_{\text{drift}}(y) = \|y - \text{sg}(y + V_p(y) - V_q(y))\|^2, \quad (1)$$

$$V_p(y) = \sum_j \frac{\kappa(y, z_j)}{Z} (z_j - y), \quad (2)$$

$$V_q(y) = \sum_k \frac{\kappa(y, y_k)}{Z'} (y - y_k), \quad (3)$$

where $\text{sg}(\cdot)$ is the stop-gradient, $\{z_j\}$ are real samples, $\{y_k\}$ are sibling generated samples, and κ is a similarity kernel. The attraction field V_p pulls y toward nearby real data; the repulsion field V_q pushes it off other generated samples to prevent mode collapse — Drifting thus trains a one-step generator with no discriminator and no iterative sampler.

Algorithm 1: Drifting force for one group of generated samples.
Input: generated batch $g \in \mathbb{R}^{C_g \times S}$, positive references $P \in \mathbb{R}^{C_p \times S}$, negative references $N \in \mathbb{R}^{C_n \times S}$, bandwidth set $\{R\}$, distance $d(\cdot, \cdot)$, neighbour counts $k_{\text{pos}}, k_{\text{neg}}$.

1. $\bar{g} \leftarrow \text{sg}(g)$; $T \leftarrow [\bar{g} \parallel N \parallel P]$ *// targets: detached self, negatives, positives*
2. $D \leftarrow d(\bar{g}, T)$ *// pairwise distance under a chosen kernel, not necessarily Euclidean*
3. $s \leftarrow \bar{D}$; $\sigma \leftarrow \max(s/\sqrt{S}, 10^{-3})$ *// data-adaptive scale from the mean pairwise distance*
4. $\hat{D} \leftarrow D/\max(s, 10^{-3})$; add a large constant to the self-diagonal of $\hat{D}$ *// block self-attraction*
5. $F \leftarrow 0$
6. **for** each bandwidth R :
 $\ell \leftarrow -\hat{D}/R$; keep only the $k_{\text{neg}}/k_{\text{pos}}$ nearest entries per row (suppress the rest)
 $a \leftarrow \sqrt{\text{softmax}_{\text{row}}(\ell) \cdot \text{softmax}_{\text{col}}(\ell)}$ *// doubly-normalized affinity (geometric mean)*
split $a = [a_{\text{neg}} \parallel a_{\text{pos}}]$; $\rho \leftarrow [-a_{\text{neg}} \sum a_{\text{pos}} \parallel a_{\text{pos}} \sum a_{\text{neg}}]$
 $f \leftarrow \rho(T/\sigma) - (\sum_x \rho_x)(\bar{g}/\sigma)$ *// attraction - repulsion, mean-centered*
 $F \leftarrow F + f/\sqrt{f^2}$ *// normalize each bandwidth to unit RMS, then sum*
7. **return** target $\text{sg}(g/\sigma + F)$; the loss for this group is $\|g/\sigma - \text{target}\|^2$.

Figure 1: Our Drifting force. The bare objective is Eq. 1; the boxed routine adds the data-adaptive scale (step 3), self-masking (step 4), nearest-neighbour pruning, the doubly-normalized geometric-mean affinity, and multi-bandwidth aggregation (step 6). The distance d in step 2 is a modular choice (Sec. 4.3).

Spatial Transformer Networks (STN). An STN [8] applies a *differentiable* affine warp to an image: from a few affine parameters θ (scale, rotation, translation) it forms a sampling grid and reads the source by bilinear interpolation, letting a network learn *where* and at what scale to place content while the whole pipeline stays end-to-end trainable. We use it as a parameter-free placement operator (it has no learned weights of its own): the generator emits one pose θ_k per glyph, and the STN warps each 32×32 glyph patch onto the shared 128×128 canvas.

4 Methodology

4.1 Overview

Given a string of $K=12$ letter classes $\{c_k\}$ and a noise vector ε , the generator G produces K single-channel layers $\{L_k\} \in [0, 1]^{128 \times 128}$ in one forward pass, each holding one placed glyph; the final canvas is their clamped sum,

$$I = \text{clamp}\left(\sum_{k=1}^K L_k, 0, 1\right). \quad (4)$$

This *sum-then-clamp* composition is deliberately the only compositing rule we allow: it contains no free per-pixel transparency mask, so the face cannot be “painted” by a hidden channel — it must be *spelled* out of real letter strokes placed by the STN. Two Drifting signals supervise the output: a per-glyph one keeps each placed image on its letter manifold, a composite one pulls I toward the real face line-drawing manifold; several structural regularizers shape the layout (Sec. 4.3). Throughout, we call the per-glyph letter signal the *glyph drift* and the composite face signal the *face drift*.

Our Drifting implementation. Algorithm 1 is our exact implementation of the drift force: it preserves the bare objective (Eq. 1) but adds several pieces, described next.

What we add on top of the original Drifting. Our implementation preserves the core of Deng et al. [4] (the doubly-normalized geometric-mean affinity, the attraction-minus-repulsion force, the multi-bandwidth aggregation, and the stop-gradient target). On top of it we introduce, and rely on, the following five modifications:

1. **A modular structural distance** (step 2). The original measures similarity by Euclidean distance; we instead allow a soft intersection-over-union distance (or several variants) for the face signal, which compares *where ink lands* rather than raw pixel differences (Sec. 4.3).

2. **Nearest-neighbour pruning.** Before forming the affinity we keep, per generated sample, only its k_{pos} nearest positive references and k_{neg} nearest negatives, so the drift target is the average of a few nearest prototypes rather than the whole collection. This preserves fine facial features that an average over the entire dataset would wash out.
3. **A data-adaptive scale** (step 3): the softmax temperature is set from the mean pairwise distance of the current batch, and inputs are divided by $\sqrt{S}$ (S = dimensionality), making the bandwidths R independent of resolution.
4. **Input noise smoothing:** before each drift evaluation we mix a little Gaussian noise into the generated, positive, and negative samples, $x \leftarrow (1 - \alpha)x + \alpha \epsilon$ with $\alpha=0.1$, which smooths the empirical score estimate.
5. **Two-level application** (per-glyph and composite), with a *categorical* same-class repulsion at the glyph level (Sec. 4.3), plus training curricula and adaptive weighting on the letter term (Sec. 4.3, 5.3).

4.2 Generator: a two-stage transformer entangled with the STN

The generator maps ε and the K letter classes $\{c_k\}$ to the K layers; it uses a token width $d=320$, $K=12$ slots, and 8 attention heads.

Tokenization. K learnable “slot” tokens (one per glyph, in the spirit of slot-based object-centric representations [12], orthogonally initialized to break permutation symmetry) are summed with a learned projection of the noise, broadcast to all slots, and a per-slot learned embedding of the slot’s letter class. Broadcasting the noise to *every* slot (rather than appending it as one extra token) matters: as a single extra token among $K+1$, the noise receives only $1/13$ of the attention and is effectively ignored, which collapses the batch-to-batch variance of the output.

Stage 1 (layout). The K slot tokens plus one noise token pass through a 3-layer pre-norm transformer encoder [19] (feed-forward width $4d$). Two linear heads read off, per slot, an affine pose $\theta_k = (r_k, \text{rot}_k, t_k^x, t_k^y)$ (scale, rotation, and translation) and a content seed s_k . Poses are bounded: $r = r_{\min} + (r_{\max} - r_{\min}) \text{sigmoid}(\cdot)$, $\text{rot} = \frac{\pi}{6} \tanh(\cdot)$, and $t^x, t^y = d_{\max} \tanh(\cdot)$. Self-attention here lets the slots negotiate placement so they tile the canvas instead of piling up.

Stage 2 (render, layout-aware). The pose is re-encoded by a small two-layer network and *added back* to each slot token, together with the content seed, the slot identity, and (again) the broadcast noise and letter embedding. A second 3-layer transformer then attends over these layout-aware tokens, so a slot rendering an “eye” already knows where its sibling eye will sit. **The two stages exist precisely to entangle the renderer with the STN:** the pose θ flows simultaneously into the affine grid that places the glyph *and* back into the rendering stage, so Stage 1 receives gradient from both the placement and the appearance.

Decoder, output sharpness, and placement. Each Stage-2 token is projected to a $4 \times 4 \times 128$ feature and upsampled by three convolutional blocks to a 32×32 glyph image $P_k = \text{sigmoid}(t_\sigma \cdot \text{logits})$, where t_σ is an output-sharpness temperature that follows a curriculum (Sec. 5.3). The glyph is warped onto the 128×128 canvas at pose θ_k by differentiable bilinear sampling on an inverse-affine grid (zero padding), giving L_k ; the layers compose by Eq. 4.

Letter conditioning (FiLM). To make each glyph *read* as its intended letter without entangling layout, we additionally condition the decoder with feature-wise linear modulation [13]: a small network maps the slot’s letter embedding to a per-channel scale and shift (γ, β) applied at each decoder block, $h \leftarrow \gamma \odot h + \beta$. The producing layer is zero-initialized so that $\gamma=1, \beta=0$ at the start of training (an identity transform), making the addition exactly backward-compatible. This conditioning route is what we ablate in Sec. 5.5.

4.3 Losses

The objective is a weighted sum of a face term, a letter term, and structural regularizers. We give the exact form of every term; ℓ_k denotes the placed ink of layer k at a pixel, I the canvas, $\overline{\text{ink}}_{\text{real}}$ the mean ink of the real collection, and $\mathbb{E}$ a mean over the indicated axes. Default weights are in Table 1.

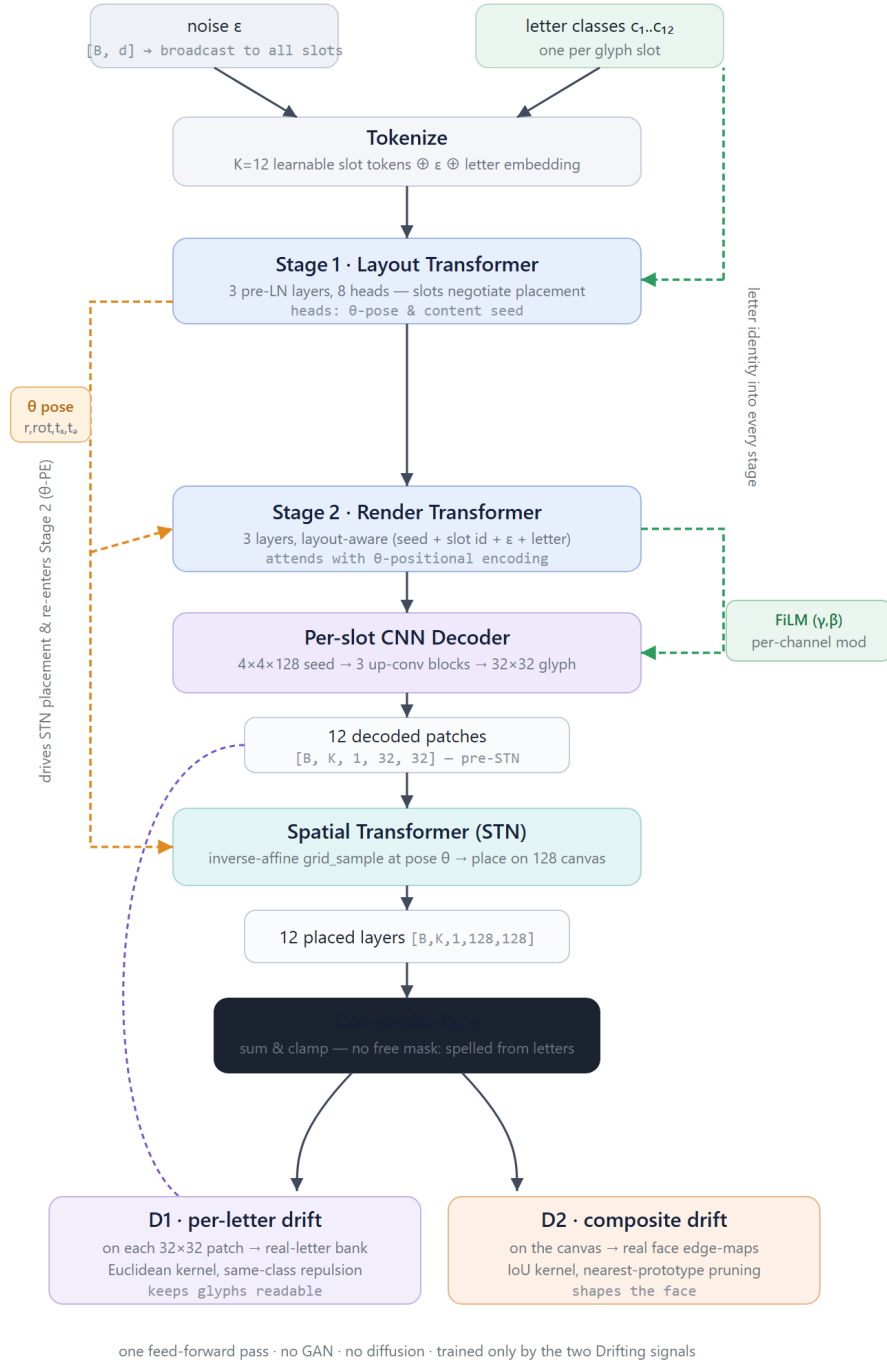


Figure 2: The generator. Noise, K per-glyph slot tokens, and the K letter classes feed a two-stage transformer; Stage 1 emits affine poses and content seeds, Stage 2 renders the glyph images that the STN places and sum-clamps into the canvas. The pose path (orange) entangles the transformer with the STN. Letter conditioning (green) enters through the embedding and the FiLM modulation. The two drift signals use different distances: Euclidean for glyphs, and Euclidean or intersection-over-union for faces.

Distances. The distance d inside Drifting (Alg. 1, step 2) is modular. The two we use most are the Euclidean distance and a soft intersection-over-union (IoU) distance,

$$d_{L2}(x, y) = \sqrt{\max(\|x\|^2 + \|y\|^2 - 2\langle x, y \rangle, 10^{-8})}, \quad (5)$$

$$d_{IoU}(x, y) = 1 - \frac{\langle x, y \rangle}{\|x\|_1 + \|y\|_1 - \langle x, y \rangle + 10^{-6}}, \quad (6)$$

the latter treating pixel values as soft occupancy (intersection = $\langle x, y \rangle$, union = $\|x\|_1 + \|y\|_1 - \text{intersection}$). We also implemented a Dice distance $(1 - 2\langle x, y \rangle / (\|x\|_1 + \|y\|_1))$, a multi-scale IoU (mean of the IoU distance over the maps average-pooled by 1, 2, 4 $\times$), a gradient-domain IoU (IoU on Sobel edge magnitudes), and a low-frequency Euclidean distance (Fourier coefficients weighted by $1/(1+10|f|)$). The IoU distance is excellent for faces but, applied to letters, collapses glyphs into region-filling strokes; we therefore use *different distances at the two levels* — Euclidean for the glyph signal (which preserves glyph shape) and Euclidean or IoU for the face signal. Our recommended model pairs an *IoU* face distance with a *Euclidean* glyph distance: IoU gives the best and most stable faces, and a strong legibility term plus a broad glyph temperature keep the placed glyphs readable enough (Sec. 5.4); the all-Euclidean variant keeps letters crisper but yields looser, noisier faces.

Face drift (composite level). We flatten the canvas to a 16384-dimensional vector, group C_g canvases together as one drift problem, and apply Alg. 1 with C_p positive and C_n negative references drawn at random from the real face line-drawings (the detached siblings within the group supply the intra-batch repulsion):

$$\mathcal{L}_{\text{face}} = \mathbb{E}[\mathcal{L}_{\text{drift}}(\text{vec}(I))], \quad \text{bandwidths } R \in \{0.005, 0.02, 0.1\}, \text{ distance } d_{\text{face}}. \quad (7)$$

Glyph drift (per-letter level, categorical repulsion). For each letter class c we gather all generated glyph images assigned to c in the batch; each is its own drift problem. Its positive references are C_p^ℓ random *real* augmented letters of the *same* class c ; its negatives are C_n^ℓ *other generated glyphs of the same class* (self excluded, detached). Thus a generated “A” is repelled only by other generated “A”s and never by a “B” — the repulsion is categorical, preventing mode collapse *within* a class without forcing different letters apart:

$$\mathcal{L}_{\text{glyph}} = \frac{1}{N} \sum_c \sum_{i \in c} \left\| P_i - \text{sg}(P_i + V_p^{(c)}(P_i) - V_q^{(c)}(P_i)) \right\|^2, \quad (8)$$

with $V_p^{(c)}$ from real class- c letters and $V_q^{(c)}$ from sibling generated class- c glyphs, Euclidean distance, the same bandwidths, normalized by the total glyph count N .

Legibility auxiliary (cross-entropy). Because Drifting matches a *distribution* and tolerates deformation, a glyph can drift far from a canonical letter shape. We add a small cross-entropy term from a *frozen* letter classifier f (Sec. 5.3), $\mathcal{L}_{\text{CE}} = \text{CE}(f(P_k), c_k)$, that pushes each glyph to be *classified* as its target letter. The classifier is trained on the *same* aggressively-augmented letters as the glyph drift, so it rewards deformation-tolerant legibility rather than upright, canonical glyphs. It is an auxiliary only: Drifting remains the main letter signal, because a bare classifier loss can be satisfied by adversarial textures.

Structural regularizers (exact forms).

$$\text{region IoU: } \mathcal{L}_{\text{rIoU}} = \lambda_{\text{rIoU}} \mathbb{E}_B \left[1 - \frac{\sum_r c_r t_r}{\sum_r c_r + \sum_r t_r - \sum_r c_r t_r + 10^{-6}} \right] \quad (9)$$

$$\text{overlap: } \mathcal{L}_{\text{ov}} = \lambda_{\text{ov}} \mathbb{E}_{B, \text{px}} \left[\frac{(\sum_k \ell_k)^2 - \sum_k \ell_k^2}{2} \right] = \lambda_{\text{ov}} \mathbb{E} \left[\sum_{i < j} \ell_i \ell_j \right] \quad (10)$$

$$\text{coverage: } \mathcal{L}_{\text{cov}} = \lambda_{\text{cov}} (\bar{I} - \overline{\text{ink}}_{\text{real}})^2 \quad (11)$$

$$\text{sharpness: } \mathcal{L}_{\text{sh}} = w_{\text{sh}} \mathbb{E}_{\text{px}} \left[1 - 4(I - \frac{1}{2})^2 \right] \quad (12)$$

$$\text{total variation: } \mathcal{L}_{\text{TV}} = \lambda_{\text{TV}} (\mathbb{E}|I_{x,y+1} - I_{x,y}| + \mathbb{E}|I_{x+1,y} - I_{x,y}|) \quad (13)$$

$$\text{locality: } \mathcal{L}_{\text{loc}} = \lambda_{\text{loc}} \mathbb{E}_{B,k} \left[\frac{\sum_p \ell_{k,p} ((y_p - c_k^y)^2 + (x_p - c_k^x)^2)}{\sum_p \ell_{k,p}} \right] \quad (14)$$

$$\text{dead-layer hinge: } \mathcal{L}_{\text{min}} = w_{\text{min}} \sum_{k=1}^K \text{ReLU}(\tau_{\text{min}} - \bar{\ell}_k), \quad \bar{\ell}_k = \mathbb{E}_{B, \text{px}}[\ell_k] \quad (15)$$

The region term is a soft (1–IoU) of the canvas and a random real face after both are coarsened into an $n \times n$ grid of mean-ink cells (pool_n averages over $128/n$ -pixel blocks); it pushes ink toward face-anatomical regions *without* encoding any specific landmark. Overlap discourages two glyphs from stacking on the same pixels (using $\sum_{i < j} \ell_i \ell_j = \frac{1}{2}[(\sum \ell)^2 - \sum \ell^2]$); coverage matches the global ink ratio; sharpness pushes pixels toward $\{0, 1\}$; total variation removes spurious thin strokes; locality keeps each glyph’s ink spatially compact (its second central moment about its own centroid); the dead-layer hinge penalizes any layer whose mean ink falls below a floor, preventing slots from going blank.

Total objective.

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{face}} + \mathcal{L}_{\text{rIoU}}}_{\text{face}} + \underbrace{w_\ell \mathcal{L}_{\text{glyph}} + w_c \mathcal{L}_{\text{CE}}}_{\text{letter}} + \mathcal{L}_{\text{ov}} + \mathcal{L}_{\text{cov}} + \mathcal{L}_{\text{sh}} + \mathcal{L}_{\text{TV}} + \mathcal{L}_{\text{loc}} + \mathcal{L}_{\text{min}}. \quad (16)$$

In the stable regime the face drift (≈ 5 – 7) dominates, the region term is ≈ 1.35 , the glyph drift ≈ 0.9 , and the cross-entropy term (large at the start, ≈ 3) is satisfied within about 100 steps and decays toward 0. The face pressure is thus roughly $5\times$ the letter pressure, which directly explains why a strong face signal bends glyphs into strokes.

Adaptive letter weighting. To control this balance precisely we optionally make w_ℓ, w_c adaptive. At each step we divide each letter term by its own current value and rescale it to a fixed target fraction of the current face pressure $F = \mathcal{L}_{\text{face}} + \mathcal{L}_{\text{rIoU}}$,

$$w_\ell = \text{clip}\left(\rho_{\text{glyph}} \frac{F}{\mathcal{L}_{\text{glyph}}}\right), \quad w_c = \text{clip}\left(\rho_{\text{CE}} \frac{F}{\mathcal{L}_{\text{CE}}}\right), \quad (17)$$

using current stop-gradient values (no moving average), so the glyph contribution is pinned to $\rho_{\text{glyph}} F$ and the cross-entropy to $\rho_{\text{CE}} F$ regardless of how the raw magnitudes drift over training. The two letter terms are balanced *independently*. We found, however, that this adaptive scheme *hurts*: it raises the letter weight precisely when the letters are already well-formed (small loss), over-constraining them and starving the face signal. Our final model therefore uses *fixed* weights (the glyph drift at its natural scale, giving a glyph-to-face ratio near 0.14, and a small fixed cross-entropy weight); fixed weights have the correct behavior automatically, since a small letter loss simply yields a small contribution (Sec. 5.4).

4.4 Data and augmentation

Both collections are unpaired. The exact pipelines follow; samples are shown in Fig. 3.

Face line-drawings (the face target). From 20,000 aligned CelebA [11] portraits we build binary line-drawings: (1) resize to 128×128 with area interpolation; (2) Gaussian blur, standard deviation 1.4 pixels; (3) Canny edge detection [2] with hysteresis thresholds (75, 170); (4) sparsify by removing connected components smaller than 8 pixels; (5) dilate once with a 3×3 elliptical kernel to thicken strokes into a clean marker-pen look. The result is binary, with a mean ink fraction of about 0.275.

Letter collection (the glyph target). From EMNIST [3] handwritten letters (26 classes) we build 2000 pre-augmented variants per class at 32×32 , in order: (1) a *coarse* elastic deformation [15] followed by a *fine* one, each a Gaussian-smoothed random displacement field renormalized to a target root-mean-square pixel amplitude and applied by warping (coarse field: smoothing ≈ 7.7 px and amplitude ≈ 2.1 px over 10 accumulation steps; fine field: smoothing ≈ 2.9 px and amplitude ≈ 0.7 px over 8 steps); (2) a center-aligned affine warp with rotation up to $\pm 35^\circ$, shear up to ± 0.25 , and independent horizontal/vertical scaling up to $\pm 18\%$ (renormalized so the longer axis is unchanged, i.e. no upscaling) — translation is *not* part of the affine; (3) a fit-to-canvas step that crops to the ink bounding box, rescales the longer side to a random 55–95% of the canvas, and recenters with a small positional jitter. With probability 0.08 a sample is drawn from an even more extreme setting (rotations to $\pm 55^\circ$, larger fields). A “mild” setting (used only for one ablation classifier) roughly halves every range. This heavy yet shape-preserving augmentation teaches the decoder a deformation-tolerant prior over each letter, which is what lets the placed glyphs bend toward facial contours while remaining recognizable; the same augmented collection trains the legibility classifier.



Figure 3: **Top:** 16 random face line-drawings (CelebA $\rightarrow$ blur $\rightarrow$ Canny $\rightarrow$ sparsify $\rightarrow$ dilate), the face target. **Bottom:** augmented letters — 8 variants each for 8 letters (EMNIST $\rightarrow$ two-stage elastic warp $\rightarrow$ affine $\rightarrow$ fit-to-canvas), the glyph target. The heavy, shape-preserving deformation lets a placed glyph bend toward a facial contour while staying recognizable.

5 Experiments

5.1 Experimental setup

Metrics. Because the data are sparse binary line-drawings, the usual Inception-based Fréchet distance is a domain mismatch. We instead report a **kernel distance in an edge-domain feature space**: we encode real and generated canvases with a frozen autoencoder trained on the face line-drawings (Sec. 5.3), reduce its bottleneck to a 256-dimensional vector by average pooling, and compute the unbiased polynomial-kernel maximum mean discrepancy (KID, $\times 10^3$, lower is better) [1] with a block estimator over 10 resamples. We complement it with improved **precision and recall** [9] (realism vs. diversity). For letters we report a frozen-classifier **legibility probe**, but stress that it is *circular* when the same classifier appears in the loss. To get a meaningful number we therefore evaluate legibility with a *held-out* classifier — one trained at a different augmentation strength than the one used in the loss — and complement it with visual inspection (Table 2, last column).

Compute. Each model trains on a single NVIDIA RTX PRO 6000 (96 GB); an 8000-step run takes about 2 hours. The full study — the configurations in Table 2 plus seed replicates and abandoned directions — used on the order of a few dozen such runs.

5.2 Baseline: pure-drifting face generation

To isolate the cost of the letter constraint — and to confirm that Drifting alone produces faces in our line-drawing domain — we first train an *unconstrained* baseline that generates face line-drawings directly, with *no letters, slots, or STN*. This baseline is itself a deliverable of the project, so we document it fully.

Architecture. A noise vector (128 dimensions) is projected by a linear layer to a $192 \times 4 \times 4$ feature map and upsampled by five nearest-neighbour convolutional blocks (each: $2 \times$ upsample, 3×3 convolution, group normalization, GELU) with channel widths $192 \rightarrow 192 \rightarrow 192 \rightarrow 96 \rightarrow 48 \rightarrow 24$, reaching 128×128 ; a final 3×3 convolution to one channel and a sigmoid produce the line-drawing. There is no autoencoder, decoder, or composition: the single output map *is* the face.

Loss and training. The face drift only, using the IoU distance, bandwidths $\{0.005, 0.02, 0.1\}$, 64 positive and 16 negative references per problem and 32 generated samples per problem, plus the same coverage, sharpness, and total-variation regularizers and the input-noise smoothing. Optimization: AdamW (learning rate 2×10^{-4} , $\beta = (0.9, 0.95)$), gradient-norm clipping at 1.0, an exponential moving average of the weights (decay 0.999, used for all sampling), batch size 256, 24000 steps, on a separate collection of 20,000 *thin* (non-dilated) Canny line-drawings (Gaussian blur 1.0, thresholds (40, 110), mean ink 0.105). The kernel distance is always measured against a model’s own training collection.

Two settings, and why they differ. We report two checkpoints of this baseline as reference points; they are trained identically and differ in *one* thing only — whether the drift target is pruned to a few nearest prototypes (Sec. 4, modification 2). The effect is instructive, because it exposes exactly what the drift force pulls a generated face toward. Recall that the attraction field V_p moves each generated sample toward an *affinity-weighted average* of the real references. *Without* pruning, that average runs over the whole collection: since the eyes, nose, and mouth sit at slightly different places across thousands of faces, averaging their line-drawings smears every feature out, and the model converges to a clean but *blurry mean face* (Fig. 4a). *With* pruning — keeping only the 4 nearest positive and 8 nearest negative references per sample — the target becomes the average of just the handful of real faces most similar to the current generation. Those few are mutually consistent, so their average *retains* sharp eyes, nose, and mouth; the price is a mild doubled-contour artifact, which appears where the nearest few faces have slightly offset edges and the average shows both (Fig. 4b). In short, pruning trades a little contour crispness against a large gain in feature sharpness by replacing a global mean target with a local, few-prototype one. The pruned setting reaches a kernel distance of about **32.6** (precision 0.55, recall 0.59), far below any letter-constrained model: it is the face-quality ceiling of the method, and the gap from it to the full W2F model is the price of spelling the face out of legible glyphs.

5.3 Frozen models used for evaluation

Two frozen models are needed to reproduce our numbers; both are trained once and then held fixed.

Feature extractor for the kernel distance. A convolutional autoencoder on 128×128 line-drawings. The encoder has three stride-2 blocks (each: 4×4 convolution, group normalization, GELU) with channel widths $1 \rightarrow 48 \rightarrow 96 \rightarrow 192$ (reducing $128 \rightarrow 16$), followed by a 1×1 convolution to a $16 \times 16 \times 16$ bottleneck; the decoder mirrors it. It is trained to reconstruct the line-drawings with a binary cross-entropy on the logits (positive-class weight 6, to counter the sparse foreground), AdamW (learning rate 2×10^{-4}), batch size 64, about 5000 steps. The kernel-distance feature is this frozen bottleneck, average-pooled to 4×4 and flattened to 256 dimensions.

Letter classifier for the legibility term. A small convolutional classifier on 32×32 glyphs: two 3×3 convolutions (32 channels) and a max-pool, two more (64 channels) and a max-pool, then two fully-connected layers ($4096 \rightarrow 128 \rightarrow 26$), with ReLU throughout. It is trained by cross-entropy

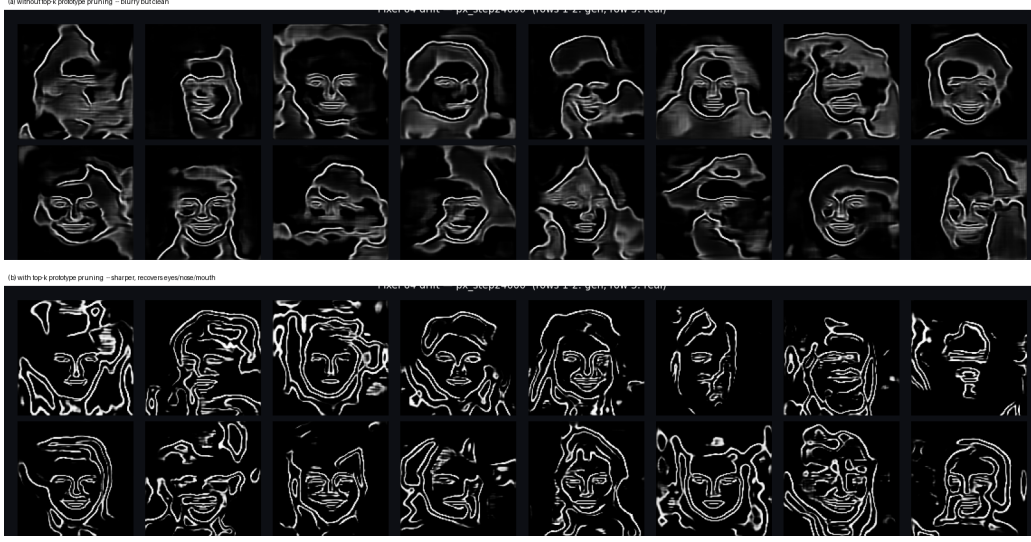


Figure 4: Unconstrained pure-drifting faces (face signal only, no letters). (a) Without nearest-neighbour pruning: clean but blurry. (b) With pruning: sharper, with recovered eyes/nose/mouth (kernel distance ≈ 32.6). This is the face-quality upper bound.

Table 1: Default hyperparameters — the recommended IoU-face configuration (kernel distance 124).

Group	Parameter	Value	Note
Model	K , token width, layers, heads	12, 320, 6 (3+3), 8	two-stage transformer
	glyph / canvas size	32 / 128	STN places glyph on canvas
Drift	$r_{\min}, r_{\max}$, rotation, $d_{\max}$	0.10, 0.40, $\pm\pi/6$, 0.6	pose bounds
	face bandwidths / pruning	0.005, 0.02, 0.1 / 4, 8	composite level
	face distance	IoU	best/most stable faces
	glyph distance / temperature	Euclidean / broad	bands 0.0125, 0.05, 0.25
	gen / pos / neg (face)	64, 64, 16	per drift problem
	pos / neg per class (glyph)	32, 32	same-class only
Letter	glyph drift weight	0.3	fixed, not adaptive
	cross-entropy weight	0.3	frozen aggressive classifier
Struct	region IoU ($n=16$), overlap, coverage	1.5, 5, 8	placement / separation
	sharpness, TV, dead-layer (τ, w)	0.5, 0.03, (.005, 150)	sharpness held constant
	locality	10^{-3}	per-glyph compactness
Optim	lr, β , EMA, grad-clip	2×10^{-4} , (.9, .95), .999, 1.0	AdamW
	batch, steps, sigmoid temp.	2048, 8000, $1.0 \rightarrow 2.5$	temp. ramps in $\rho=t/T$

with Adam (learning rate 10^{-3}), batch size 256, about 2000 steps, on the augmented letter collection (with a light ± 2 -pixel shift); the heavy deformation comes from the collection itself. We train two versions that differ *only* in the augmentation strength of their training letters, and use the one matched to the aggressively-augmented letters: a classifier trained on mildly-augmented letters drags the generated glyphs toward upright, canonical shapes and fights the deformation that the glyph drift is trying to produce. On its own held-out aggressively-augmented letters the matched classifier reaches 64% accuracy — low in absolute terms (the augmentation is severe) but adequate as a soft legibility signal.

Training details and curricula. AdamW (learning rate 2×10^{-4} , $\beta = (0.9, 0.95)$), gradient-norm clipping at 1.0, an exponential moving average of the weights (decay 0.999, used for all evaluation), batch size 2048, for 8000 steps. One quantity follows a curriculum: the output-sharpness temperature ramps linearly $1.0 \rightarrow 2.5$ in normalized progress $\rho = t/T$ (applied to both the live and the averaged weights so the averaged samples never lag); the sharpness-regularizer weight (0.5) and the letter weight (0.5) are held constant. We checkpoint every 2000 steps and *keep every checkpoint*, because

Table 2: Benchmark. Edge-domain face kernel distance ($\times 10^3$, lower better), improved precision/recall, and a held-out letter-legibility accuracy — a frozen classifier trained at a *different* augmentation strength than the one in the loss, so it is not circular (higher better). All at the early-peak checkpoint, 4096 generated samples. **Top:** the pure-face ceiling and the FiLM $\times$ face-distance ablation. **Bottom:** the letter-drift temperature sweep (Euclidean face +FiLM; the sharp row is a 5-seed mean).

Configuration	Kernel dist. $\downarrow$	prec $\uparrow$	rec $\uparrow$	letter acc $\uparrow$
Pure-face baseline (no letters, no STN)	32.6	0.55	0.59	—
Euclidean face, no FiLM	169	0.92	0.11	0.53
Euclidean face, +FiLM	150 ± 18	0.84	0.20	0.52
IoU face, +FiLM (<i>recommended</i>)	124 ± 5	0.88	0.15	0.38
<i>Euclidean face +FiLM, varying letter-drift temperature:</i>				
sharp (5 seeds)	150 ± 18	0.85	0.18	0.53
medium	140	0.87	0.18	0.58
broad (best legibility)	152	0.80	0.22	0.66

the model peaks early (around step 6000) and then slowly over-trains; we select the best checkpoint by the kernel distance rather than taking the last.

5.4 Results and benchmarking

The legibility–realism trade-off (main finding). Every lever moves the model along a single front: better faces cost legibility, and vice versa. We characterize four levers.

Distance. A structural IoU face distance (with Euclidean glyphs) gives the best and most *stable* faces (124 ± 5 over seeds, versus 150 ± 18 for the all-Euclidean model), but, caring only about *where* ink lands, bends the glyphs into face-conforming arcs (legibility 0.38); IoU at *both* levels melts them entirely. Euclidean glyphs keep letters crisp at a higher, noisier face distance.

Letter-drift temperature. The softmax bandwidth of the glyph drift (Alg. 1) is a clean, previously-unexplored knob. Its effect on the *letters* is robust — a *sharp* temperature locks each glyph onto its single nearest prototype (legibility 0.53); a *broad* one averages many into a generic, more-readable glyph (0.66) — while its effect on the *face* distance is within seed noise (150 ± 18 for the sharp setting). The reliable lever is the legibility we trade, not the face number.

Legibility weight. The best cross-entropy weight depends on the face distance. Under the Euclidean model a small weight (0.15) is the sweet spot — less noisy than 0.3, not as weak as 0.1 (which is worse on faces). Under the IoU model the strong face pressure bends glyphs harder, so a larger weight (0.3) is needed to keep them legible; this is the value in our recommended configuration.

Pushing letters harder backfires. Rescaling the letter contribution *up* to a fixed, larger fraction of the face pressure is strictly harmful: at glyph-to-face ratios 0.3, 0.6, 1.0 the faces collapse (kernel distances 918, 703, 1474) *and* the glyphs grow noisier. The operating point near a ratio of 0.14 is essentially the ceiling of letter pressure the faces tolerate.

5.5 Ablation: letter conditioning (FiLM)

Our main controlled ablation turns the feature-wise letter conditioning on and off with everything else fixed (Euclidean distances, the matched legibility classifier, identical seeds and schedule). Conditioning improves the face kernel distance by about 20 points and roughly doubles recall (0.22 vs. 0.11): it both sharpens structure and counters mode collapse; without it the model reaches higher precision but markedly lower diversity (Table 2). On the letters themselves the effect is smaller — both variants produce the deformed, heavily-augmented style — with conditioning giving slightly more consistent internal structure. Figures 8 and 9 show the Euclidean model *with* and *without* feature-wise conditioning; with FiLM off the faces are visibly less coherent.



Figure 5: W2F samples (IoU face distance, broad letter temperature, legibility weight 0.3). Each row is one sample: **(left)** the composite face; **(middle)** the same face colored by source layer; **(right)** the $K=12$ letters, each shown as its decoded 32×32 patch before the spatial transformer (left) and its colored placement on the canvas after the transformer (right). Part 1 of 3.



Figure 6: W2F samples, part 2 of 3. Layout as in Fig. 5.

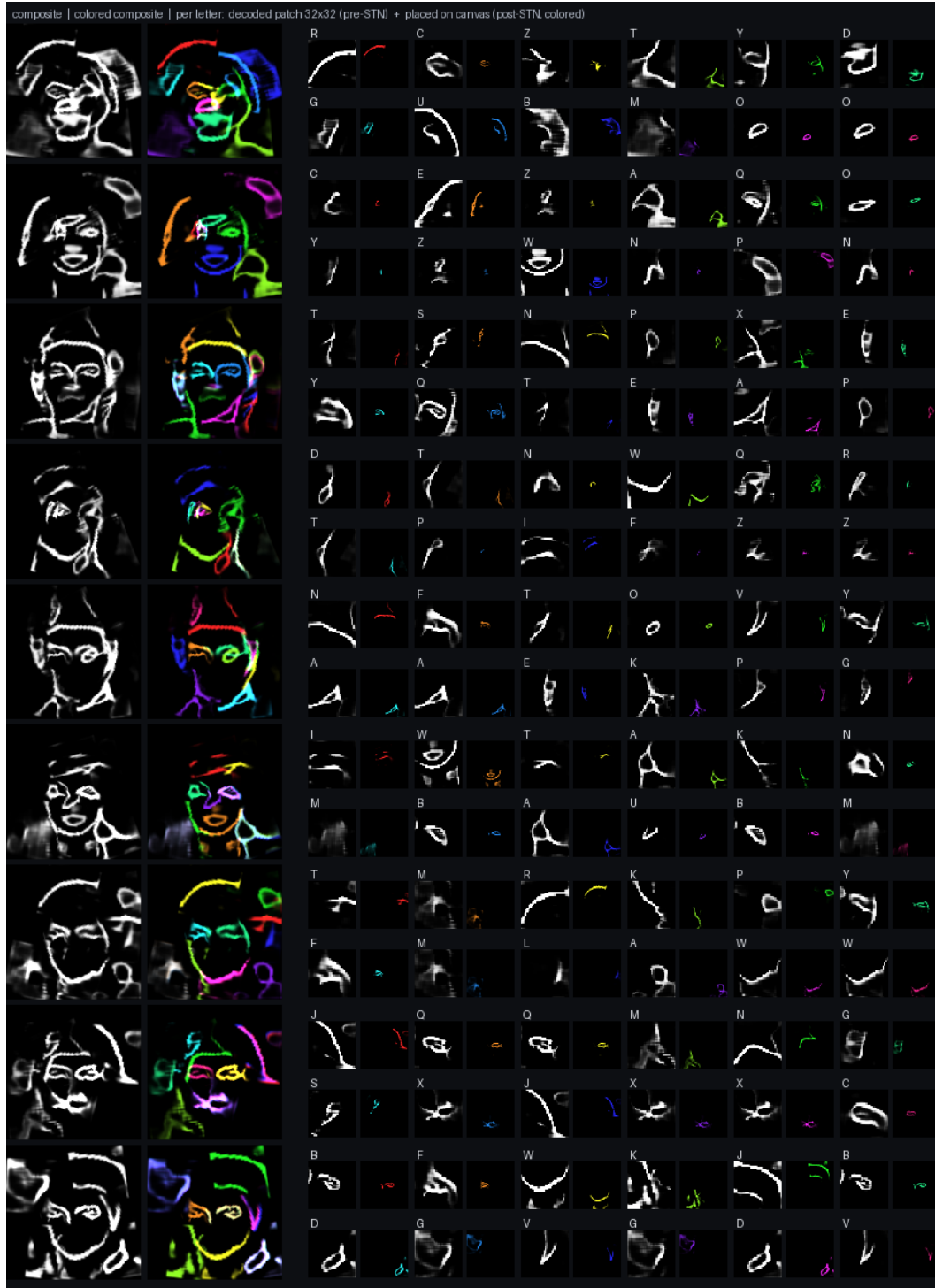


Figure 7: W2F samples, part 3 of 3. Layout as in Fig. 5.

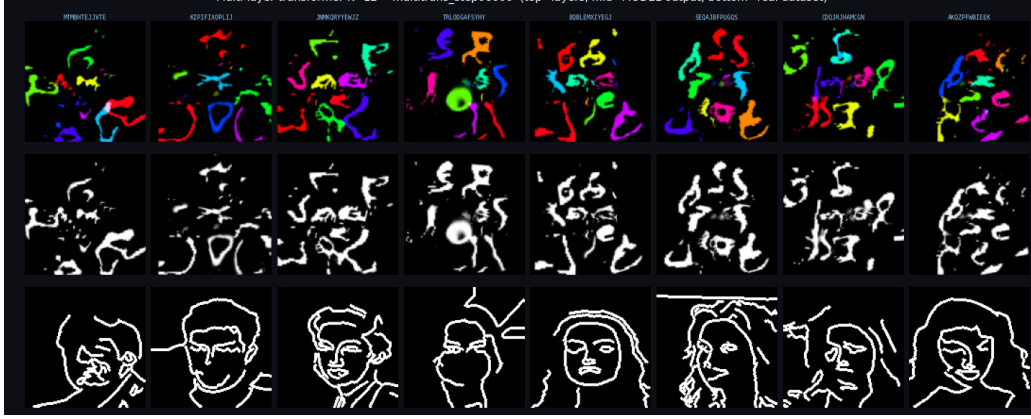


Figure 8: Euclidean distances *with* FiLM (sharp letter temperature, legibility weight 0.15; best of five seeds, kernel distance 122.6). **Top:** the $K=12$ per-glyph layers (colored by slot). **Middle:** the composited face. **Bottom:** a real line-drawing for reference. The FiLM-off counterpart is Fig. 9.

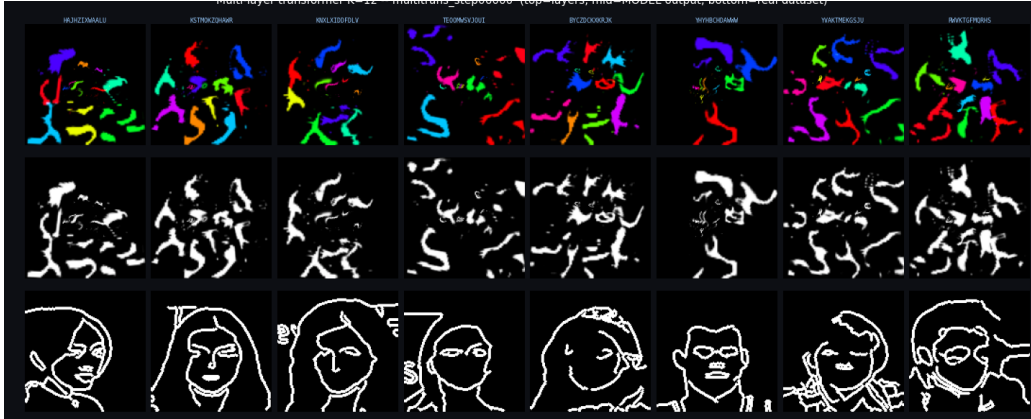


Figure 9: Ablation — Euclidean distances, *no* feature-wise conditioning (no FiLM, no IoU). Same three-row layout as Fig. 8 (colored layers / composite / real). Removing FiLM weakens facial structure relative to the FiLM-enabled Euclidean model.

Secondary observations. (i) The legibility classifier must match the augmentation strength of the glyph signal — a classifier trained on mild augmentation over-canonicalizes the output. (ii) The cross-entropy term is satisfied almost immediately and contributes little magnitude after warm-up; once its raw value has collapsed it cannot be revived by reweighting (amplifying a near-zero loss only injects noise), consistent with its admitting near-adversarial solutions, so we keep it a small auxiliary. (iii) The model peaks early and over-trains, which is why we select the best checkpoint.

6 Limitations

(i) The legibility–realism trade-off is fundamental to our setup and not fully resolved; the best single configuration is a compromise. (ii) The letter-legibility metric is a circular proxy and over-states readability. (iii) Results show non-trivial seed variance, so we report best-checkpoint numbers. (iv) The number of glyphs and the canvas size are fixed; arbitrary-length strings and higher resolutions are future work. (v) As face synthesis, the method shares general misuse considerations, mitigated here by the abstract, glyph-built, edge-only outputs.

7 Conclusion

W2F shows that a one-step generator, supervised only by a particle-based Drifting objective with no face priors and no paired data, can compose human faces out of letter glyphs. A two-stage transformer co-adapted with a spatial transformer provides the layout, feature-wise conditioning injects letter identity, and two-level Drifting — with different distances for letters and faces — plus a small legibility term balances “looks like a letter” against “looks like a face”. The remaining tension between legibility and realism is, we argue, the core scientific content of the problem, and we characterize the levers — distance choice, letter weight, and classifier matching — that move along it.

Acknowledgments and Disclosure of Funding

The implementation and a draft of this paper were produced with Claude Code (Anthropic), under the authors’ direction; the authors designed the experiments, made all research decisions, and verified the reported results.

References

- [1] Mikołaj Bińkowski, Danica J. Sutherland, Michael Arbel, and Arthur Gretton. Demystifying MMD GANs. In *International Conference on Learning Representations (ICLR)*, 2018.
- [2] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 8(6):679–698, 1986.
- [3] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and Andre van Schaik. EMNIST: an extension of MNIST to handwritten letters. In *International Joint Conference on Neural Networks (IJCNN)*, 2017.
- [4] Mingyang Deng, He Li, Tianhong Li, Yilun Du, and Kaiming He. Generative modeling via drifting. *arXiv preprint arXiv:2602.04770*, 2026.
- [5] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [7] Shir Iluz, Yael Vinker, Amir Hertz, Daniel Berio, Daniel Cohen-Or, Ariel Shamir, and Lingjie Liu. Word-as-image for semantic typography. *ACM Transactions on Graphics (SIGGRAPH)*, 42(4), 2023.
- [8] Max Jaderberg, Karen Simonyan, Andrew Zisserman, and Koray Kavukcuoglu. Spatial transformer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [9] Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [10] Yujia Li, Kevin Swersky, and Richard Zemel. Generative moment matching networks. In *International Conference on Machine Learning (ICML)*, 2015.
- [11] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *International Conference on Computer Vision (ICCV)*, 2015.
- [12] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [13] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence*, 2018.
- [14] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. DreamFusion: Text-to-3D using 2D diffusion. In *International Conference on Learning Representations (ICLR)*, 2023.
- [15] Patrice Y. Simard, Dave Steinkraus, and John C. Platt. Best practices for convolutional neural networks applied to visual document analysis. In *International Conference on Document Analysis and Recognition (ICDAR)*, 2003.

- [16] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [17] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning (ICML)*, 2023.
- [18] Maham Tanveer, Yizhi Wang, Ali Mahdavi-Amiri, and Hao Zhang. DS-Fusion: Artistic typography via discriminated and stylized diffusion. In *International Conference on Computer Vision (ICCV)*, 2023.
- [19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [20] Xuemiao Xu, Linling Zhang, and Tien-Tsin Wong. Structure-based ASCII art. *ACM Transactions on Graphics (SIGGRAPH)*, 29(4), 2010.

A Detailed experiment configurations

All five models share the base recipe of Sec. 5.1 and Table 1 (two-stage transformer, $K=12$, multi-bandwidth drift, AdamW, EMA, sum-then-clamp composition); the pure-face baselines drop the letters, slots, and STN entirely and generate the 128×128 line-drawing directly (Sec. 5.2). Tables 3 and 4 give the full configurations of, respectively, the three letter-constrained models and the two pure-face baselines.

Reported numbers use the early-peak checkpoint selected by the kernel distance. Rows whose value is shared by all columns are centered across them; the rows that distinguish the configurations are in bold.

Table 3: The three letter-constrained models (Sec. 5.4): the recommended IoU-face model and the two ablations.

	Recommended (IoU+FiLM)	Ablation (Euclid. +FiLM)	Ablation (Euclid., no FiLM)
slots K / token width		12 / 320	
transformer layers / heads		6 (3+3) / 8	
glyph / canvas size		32 / 128	
pose bounds r , rot , $d_{\max}$		[0.10, 0.40], $\pm\pi/6$, 0.6	
face distance	IoU	Euclidean	Euclidean
face top-k (pos,neg)	4, 8	—	—
glyph distance		Euclidean	
glyph temperature	.0125, .05, .25	.0025, .01, .05	.005, .02, .1
face bandwidths		0.005, 0.02, 0.1	
FiLM conditioning	yes	yes	no
cross-entropy weight	0.3	0.15	0.3
glyph drift weight (fixed)		0.3	
gen / pos / neg (face)		64 / 64 / 16	
pos / neg per class (glyph)		32 / 32	
region-IoU / overlap / cov		1.5 / 5 / 8	
sharpness / TV / locality		0.5 / 0.03 / 10^{-3}	
sigmoid temp. (curriculum)		1.0 $\rightarrow$ 2.5	
optimizer		AdamW	
learning rate		2×10^{-4}	
β_1, β_2		0.9, 0.95	
gradient clip (norm)		1.0	
EMA decay		0.999	
batch (generated / step)		2048	
steps (peak $\approx$ 6000)		8000	

Table 4: The two pure-face baselines (Sec. 5.2): a convolutional generator with no letters, slots, or STN, trained with the face drift only on thin (non-dilated) Canny line-drawings.

	no pruning	top- k pruning
generator	noise $\rightarrow$ fc $\rightarrow$ 5 nearest-upsample conv blocks	
base width / noise dim	192 / 128	
channel widths	192, 192, 192, 96, 48, 24	
output	128 $\times$ 128 sigmoid map	
face distance	IoU	
face top-k (pos,neg)	none	4, 8
bandwidths	0.005, 0.02, 0.1	
gen / pos / neg	32 / 64 / 16	
coverage / sharpness / TV	5 / 1.0 / 0.03	
input-noise smoothing	0.1	
optimizer	AdamW	
learning rate	2×10^{-4}	
β_1, β_2	0.9, 0.95	
gradient clip (norm)	1.0	
EMA decay	0.999	
batch	256	
steps	24000	
training data	thin Canny edges (blur 1.0, thresholds 40/110)	